

# A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Nguyễn Vũ Thái<sup>1</sup> and Bùi Văn Dũng<sup>2,3</sup>

<sup>1</sup> University of Transport and Communications, Ho Chi Minh City Campus, Vietnam  
nvthai@utc2.edu.vn

<sup>2</sup> Vietnam National University Ho Chi Minh City, Vietnam

<sup>3</sup> University of Information Technology, Vietnam  
dungbv.17@grad.uit.edu.vn

**Abstract** Deploying machine-learning-based intrusion detection on resource-constrained edge devices requires balancing accuracy, latency, and operational cost. We present a split-deployment IDS architecture across two NVIDIA Jetson Orin Nano Super devices connected through Apache Kafka [9]. The system implements a two-stage pipeline: (1) a lightweight autoencoder anomaly gate on Jetson #1 filters benign traffic, and (2) a PySpark `PipelineModel` classifier on Jetson #2 processes only suspicious flows. Central infrastructure (Kafka, PostgreSQL, InfluxDB, Grafana) runs on a host PC. The classifier is the model selected by our companion Spark ML study under a *leakage-aware* evaluation (label-leaking port features removed, feature-level duplicate flows eliminated). We evaluate three distributed modes—pipeline split, horizontal Kafka consumer scaling, and a Spark standalone cluster—streaming CICIDS2017 network flows [17]. Results show that the anomaly gate reduces Spark inference load while preserving detection accuracy, with **[p95]** ms p95 batch latency on Jetson Orin Nano Super under **[load]** flows/s.

**Keywords:** Intrusion detection · Edge computing · Jetson Orin Nano Super · Kafka · PySpark · Stream processing.

## 1 Introduction

Network intrusion detection systems (IDS) increasingly must operate closer to data sources to reduce bandwidth and response time [18]. Edge devices such as the NVIDIA Jetson Orin Nano Super offer limited CPU, memory, and thermal headroom, making monolithic ML inference challenging. A practical edge IDS must therefore decide *which* model to deploy (small, accurate, explainable) and *how* to distribute inference across nodes under streaming load.

**Contributions.** (1) A two-stage distributed IDS combining a lightweight anomaly gate with a Spark-based classifier; (2) Three deployment modes for a two-node Jetson cluster (pipeline split, horizontal scaling, Spark standalone cluster); (3) An end-to-end performance evaluation reporting throughput, latency percentiles, and per-node resource usage—metrics offline studies omit; (4) Open, reproducible scripts integrated with a *leakage-aware* Spark ML training pipeline, so the deployed model is the one validated under a rigorous offline protocol.

## 2 Related Work

Edge computing pushes analytics closer to data sources to reduce latency and bandwidth [18, 6]. Prior edge IDS studies [16, 3, 2] benchmark single-node inference on resource-constrained boards—including recent NVIDIA Jetson deployments—but rarely evaluate multi-node Kafka pipelines with explicit role separation. Streaming platforms such as Kafka [9] decouple producers and consumers; however, most ML-based IDS papers [8, 5, 19] report offline batch accuracy without end-to-end throughput or per-node resource traces.

**Split inference.** Anomaly-detection surveys [4, 15] motivate two-stage designs: a lightweight gate followed by a heavier classifier. Autoencoder-based detectors [11, 7] and federated edge IDS [13] explore similar ideas on servers or homogeneous edge nodes—and, recently, on a single Jetson Nano [7]—but without PySpark MLlib classifiers on NVIDIA Jetson Orin Nano Super [14]. Our companion ML study selects a leak-free SHAP Top-30 model on CICIDS2017 [17, 12]; this paper focuses on *how* to deploy that model across two Jetsons with a measurable gate-skip ratio and three scaling modes. Table 1 positions our system against representative related work.

**Table 1.** Comparison with related edge and streaming IDS deployments.

| Work                     | Focus                       | Platform         | Streaming    | Our difference                           |
|--------------------------|-----------------------------|------------------|--------------|------------------------------------------|
| Shi et al. [18]          | Edge vision                 | Generic          | —            | Kafka + PySpark IDS on Jetson            |
| Sforzin et al. [16]      | Snort IDS on board          | RPi / edge       | Batch        | End-to-end Kafka replay + 3 deploy modes |
| Hamdan et al. [6]        | Edge IoT survey             | Generic          | Partial      | Measured 2-node Jetson cluster           |
| Nguyen et al. [13]       | Federated IDS               | IoT gateways     | —            | Split anomaly gate + classifier          |
| Kreps et al. [9]         | Log streaming               | Server           | Kafka        | Suspicious-flow topic forwarding         |
| Alrawashdeh & Purdy [1]  | Deep-learning IDS           | Workstation      | Near-line    | Lightweight AE gate on Jetson            |
| López-Martín et al. [11] | AE detection                | GPU server       | Offline      | AE on gate, Spark only on suspicious     |
| NVIDIA Jetson [14]       | Embedded GPU                | Jetson Orin      | —            | Role-based pipelines + Influx metrics    |
| <b>This work</b>         | <b>Distributed edge IDS</b> | <b>2× Jetson</b> | <b>Kafka</b> | <b>Modes A/B/C + open scripts</b>        |

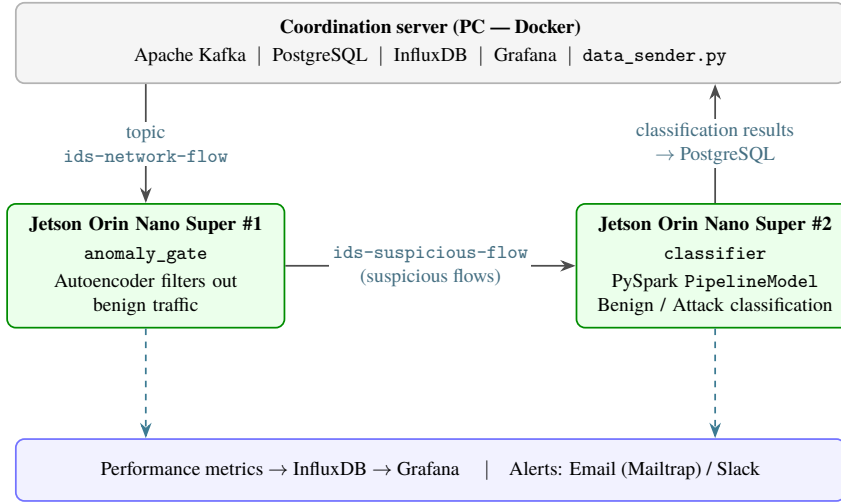


Figure 1. Split-deployment architecture (PC + 2× Jetson Orin Nano Super).

### 3 System Architecture

#### 3.1 Split Deployment Overview

Figure 1 shows the split-deployment architecture. A host PC runs Kafka, PostgreSQL, InfluxDB, and Grafana. Two Jetson Orin Nano Super nodes consume network-flow messages from topic `ids-network-flow`. In pipeline-split mode, Jetson #1 runs the anomaly gate and forwards only *suspicious* flows to topic `ids-suspicious-flow`; Jetson #2 then applies the PySpark classifier and writes classification results back to PostgreSQL. Both nodes stream performance metrics to InfluxDB/Grafana and raise alerts via Email/Slack (dashed arrows in Fig. 1).

#### 3.2 Pipeline Roles

- **anomaly\_gate**: sklearn autoencoder + scaler; MSE threshold calibrated offline (Exp 8) on benign training flows only.
- **classifier**: PySpark PipelineModel with the leak-free SHAP-selected feature set.
- **full**: combined pipeline for horizontal scaling via Kafka consumer groups.

#### 3.3 Monitoring and Alerting

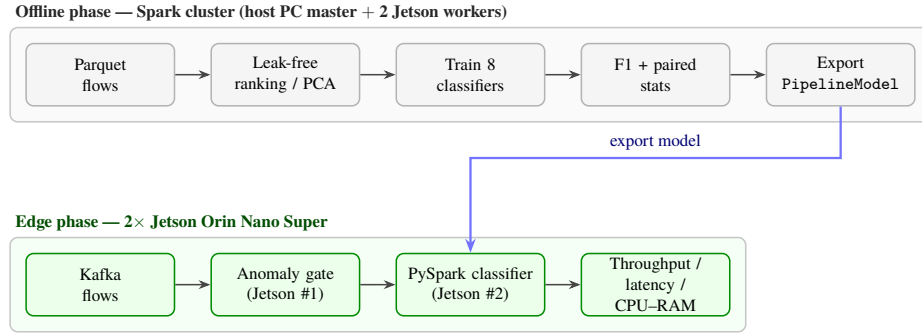
Each node tags metrics with `EDGE_NODE_ID` in InfluxDB and PostgreSQL. Alerts are emitted from a designated primary node only, avoiding duplicate notifications.

## 4 Implementation

Training and model export run on a Spark standalone cluster (host PC as master, two Jetson Orin Nano Super devices as workers) using Apache Spark ML [20]. The edge stack is implemented in Python (`jetson/edge/`) with configurable roles via environment variables. Every offline run logs its configuration (random seed, leakage/dedup switches, library versions) so each exported model is traceable to the exact pipeline that produced it.

## 5 From Spark Training to Edge Deployment

This paper does *not* compare Spark against Jetson Orin Nano Super as competing platforms. The two components address complementary phases of the same train–deploy pipeline (Fig. 2). Spark answers *which model and how many features* achieve acceptable accuracy on the full CICIDS2017 dataset [17]; Jetson Orin Nano Super answers *whether that model can run in real time* under memory, thermal, and bandwidth constraints at the network edge [18].



**Figure 2.** Two-phase train–deploy pipeline linking Spark ML experiments to Jetson edge inference.

### 5.1 Offline Phase (Spark ML), Leakage-Aware

In our companion ML study, four feature configurations—baseline (all features), RF Importance Top-30, SHAP Top-30, and PCA ( $k=40$ )—are evaluated under a fixed 80/20 train–test split with eight Spark ML classifiers and statistical validation (repeated paired resampling and permutation tests). PCA uses  $k=40$  rather than 30 because it *extracts* components (linear combinations spreading variance across many axes) rather than *selecting* original features, so it needs more components to retain comparable variance; across the common  $\{20, 30, 40\}$  sweep,  $k=40$  retains the most variance and is the fairest representative operating point for PCA. LightGBM is excluded because its SynapseML

native libraries are x86\_64-only and cannot run on the ARM64 Jetson Orin Nano Super deployment target. Crucially, the offline protocol is *leakage-aware*—that is, it removes information that would not be available at deployment time but otherwise inflates test scores. Motivated by documented labelling and duplication errors in CICIDS2017 [10], the label-leaking `destination_port` feature is removed and flows that are identical on all feature columns are de-duplicated before splitting. This yields realistic F1 values (Table 2) rather than the near-perfect scores reported when these leaks are left in. Offline metrics constrain *which* exported model is worth deploying; they are not directly comparable to edge throughput or latency.

**Table 2.** Bridge table: offline Spark selection criteria (leak-free Exp 7 on CICIDS2017). Fill from `cross_method_summary.csv`.

| Configuration             | F1            | Features     | Size (MB)   | Edge rationale                         |
|---------------------------|---------------|--------------|-------------|----------------------------------------|
| Baseline + <b>[m1]</b>    | <b>[f1_1]</b> | <b>[nf1]</b> | <b>[s1]</b> | Reference accuracy; smallest model     |
| SHAP Top-30 + <b>[m3]</b> | <b>[f1_3]</b> | <b>[nf3]</b> | <b>[s3]</b> | Explainable; fewer features (deployed) |
| RF Top-30 + <b>[m2]</b>   | <b>[f1_2]</b> | <b>[nf2]</b> | <b>[s2]</b> | Same feature count; embedded ranking   |
| PCA $k=40$ + <b>[m4]</b>  | <b>[f1_4]</b> | <b>[nf4]</b> | <b>[s4]</b> | Unsupervised reduction                 |

## 5.2 Edge Phase (Jetson Orin Nano Super)

The exported `PipelineModel` (`VectorAssembler`  $\rightarrow$  `StandardScaler`  $\rightarrow$  classifier) is loaded on Jetson #2 with only the selected SHAP features, reducing assembly and inference cost relative to the  $\approx 78$ -feature baseline. A lightweight sklearn autoencoder gate (Exp 8) on Jetson #1 filters benign flows before they reach Spark inference, further lowering load on the classifier node. End-to-end edge evaluation measures throughput (flows/s), latency p50/p95 (ms), per-node CPU/RAM/temperature, attack-class F1, and gate-skip ratio—metrics that offline Spark experiments do not capture.

## 5.3 Division of Evaluation Metrics

Table 3 clarifies which metric belongs to which phase. Mixing offline F1 with edge latency in a single ranking would conflate accuracy with operational feasibility.

We deploy SHAP Top-30 (with the classifier chosen in Exp 7) on Jetson #2 because offline results show near-baseline F1 with substantially fewer features and an acceptable model footprint. Section 7 validates this choice under streaming load; the companion ML paper reports the full four-method comparison and drift analysis.

# 6 Experimental Setup

## 6.1 Hardware and Software

2  $\times$  NVIDIA Jetson Orin Nano Super Developer Kit (6-core Arm Cortex-A78AE, NVIDIA Ampere GPU with 1024 CUDA + 32 Tensor cores, 8 GB LPDDR5, 256 GB NVMe SSD,

**Table 3.** Metric split between offline Spark training and edge Jetson inference.

| Aspect            | Spark (offline)                          | Jetson (edge)                         |
|-------------------|------------------------------------------|---------------------------------------|
| Primary goal      | Classification accuracy                  | Real-time feasibility                 |
| Main metrics      | F1, AUC-PR, Precision/Recall             | Throughput, latency p50/p95           |
| Secondary metrics | Train time, batch pred. time, model size | CPU, RAM, temperature, gate skip %    |
| Data              | CICIDS2017 train/test split              | Kafka replay at <b>[rate]</b> flows/s |
| Scale             | Millions of flow records                 | 2 nodes, 8 GB RAM each                |

~67 TOPS), host PC running Docker Compose, WiFi LAN. PySpark **[spark]**, Apache Kafka **[kafka]**, InfluxDB and Grafana for metrics.

## 6.2 Workloads

CSV replay through `data_sender.py` at **[rate]** flows/s over CICIDS2017. Reported metrics: throughput, p50/p95 latency, per-node CPU/RAM/temperature, attack-class F1, and gate-skip ratio.

## 6.3 Baselines

Single-node full pipeline vs. distributed modes A (pipeline split), B (horizontal Kafka scaling), and C (Spark standalone cluster).

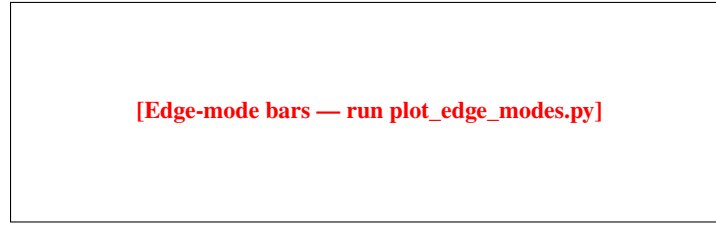
## 7 Results

We run `run_benchmarks.sh` on the Jetson hardware in three phases (`local`, `run`, `merge`). End-to-end distributed metrics are exported to `summary.csv` and filled into Table 4.

**Table 4.** Distributed deployment comparison (fill from `summary.csv` after benchmarks).

| Mode              | Throughput (flows/s) | Latency p95 (ms) | Gate skip (%) | Energy/inf. (mJ) | Attack F1    |
|-------------------|----------------------|------------------|---------------|------------------|--------------|
| Single node       | <b>[tp0]</b>         | <b>[lat0]</b>    | <b>[gs0]</b>  | <b>[e0]</b>      | <b>[f10]</b> |
| A: Pipeline split | <b>[tpA]</b>         | <b>[latA]</b>    | <b>[gsA]</b>  | <b>[eA]</b>      | <b>[f1A]</b> |
| B: Horizontal     | <b>[tpB]</b>         | <b>[latB]</b>    | N/A           | <b>[eB]</b>      | <b>[f1B]</b> |
| C: Spark cluster  | <b>[tpC]</b>         | <b>[latC]</b>    | N/A           | <b>[eC]</b>      | <b>[f1C]</b> |

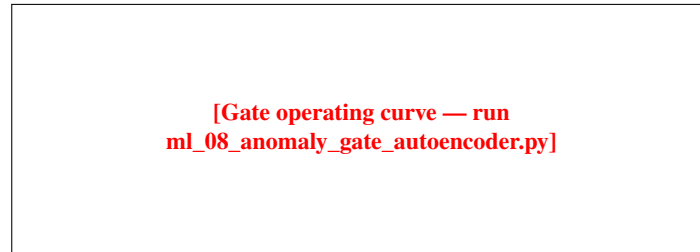
**Energy efficiency.** On-board power is sampled with `tegrastats` during each run; the average rail power is integrated over the window to obtain energy, and divided by the number of classified flows to report energy-per-inference (mJ). Because the anomaly gate offloads a **[gsA]**% fraction of benign traffic before the heavier Spark classifier runs,



**Figure 3.** Throughput and p95 latency across deployment modes (from `summary.csv`).

the pipeline-split mode lowers energy-per-inference relative to the single-node baseline (Table 4) — a saving that latency and throughput alone do not expose.

The anomaly gate is a *tunable* filter rather than a single point: sweeping its threshold trades attack recall against the fraction of benign flows offloaded from Spark (Fig. 4). This lets an operator pick an operating point that fits the available edge budget.



**Figure 4.** Anomaly-gate operating curve: attack recall vs. gate-skip ratio (and FPR).

## 8 Discussion

Pipeline split offloads Spark from the majority of benign traffic; the realized benefit scales with the gate-skip ratio (`[gsA]`% in our runs). Horizontal scaling improves throughput when Kafka partitions  $\geq 2$ . Spark standalone cluster mode adds coordination overhead on the Orin Nano Super and is most useful when model or batch size exceeds single-node memory.

As summarized in Sec. 5, offline Spark experiments and edge Jetson benchmarks answer different questions. Feature reduction from  $\approx 78$  to 30 dimensions (SHAP Top-30) directly reduces `VectorAssembler` and `PipelineModel` cost on the Orin Nano Super, while the anomaly gate skips a large fraction of benign flows before Spark inference is invoked. Together, these design choices connect the accuracy-oriented conclusions of the companion ML study to a deployable, measurable edge architecture.

### 8.1 Limitations

Benchmarks are obtained on a two-node lab cluster over WiFi; wired networking and larger clusters may shift latency tails. The anomaly-gate threshold trades attack recall against gate-skip ratio; we characterize this as an operating curve (Fig. 4) rather than a single point, leaving live-traffic re-calibration of the gate to future work. Finally, results are tied to CICIDS2017 replay; live-traffic validation remains future work.

## 9 Conclusion

We presented a reproducible distributed edge IDS for Jetson Orin Nano Super clusters with Kafka streaming and PySpark inference, deploying a model validated under a leakage-aware offline protocol. Future work includes TensorRT optimization, live-traffic validation, and federated retraining under drift.

**Acknowledgments.** This work was conducted at the University of Transport and Communications (Ho Chi Minh City Campus) and the University of Information Technology, Vietnam National University Ho Chi Minh City, as part of a graduation thesis research project.

**Disclosure of Interests.** The authors have no competing interests to declare relevant to this article.

## References

1. Alrawashdeh, K., Purdy, C.: Toward an online anomaly intrusion detection system based on deep learning. In: Proc. 15th IEEE Int. Conf. on Machine Learning and Applications (ICMLA). pp. 195–200 (2016). <https://doi.org/10.1109/ICMLA.2016.0040>
2. Asghar, M.Z., et al.: HED-ID: an edge-deployable and explainable intrusion detection system optimized via metaheuristic learning. *Scientific Reports* **16**, 2313 (2026). <https://doi.org/10.1038/s41598-025-32183-8>
3. Baalaji, K., et al.: Deep learning-based ensemble stacking for enhanced intrusion detection in IoT-edge platforms. *Discover Applied Sciences* **7**, 928 (2025). <https://doi.org/10.1007/s42452-025-06871-z>
4. Chandola, V., Banerjee, A., Kumar, V.: Anomaly detection: A survey. In: *ACM Computing Surveys*. vol. 41, pp. 1–58 (2009)
5. Ferrag, M.A., Maglaras, L., Janicke, H., Jiang, J., Shu, L.: Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *Journal of Information Security and Applications* **50**, 102419 (2020)
6. Hamdan, S., Ayyash, M., Almajali, S.: Edge-computing architectures for internet of things applications: A survey. *Sensors* **20**(22), 6441 (2020). <https://doi.org/10.3390/s20226441>
7. Hasan, T., Hossain, A., Ansari, M.Q., Syed, T.H.: Enhanced intrusion detection in IIoT networks: A lightweight approach with autoencoder-based feature learning (2025), arXiv:2501.15266
8. Khraisat, A., Gondal, I., Vamplew, P., Kamruzzaman, J.: Survey of intrusion detection systems: techniques, datasets and challenges. *Cybersecurity* **2**(1), 1–22 (2019)
9. Krepes, J., et al.: Kafka: a distributed messaging system for log processing. In: *NetDB* (2011)



10. Lanvin, M., Gimenez, P.F., Han, Y., Majorczyk, F., Mé, L., Totel, É.: Errors in the CIDS2017 dataset and the significant differences in detection performances it makes. In: Risks and Security of Internet and Systems (CRiSIS 2022), Revised Selected Papers. pp. 18–33. Lecture Notes in Computer Science, Springer (2023). [https://doi.org/10.1007/978-3-031-31108-6\\_2](https://doi.org/10.1007/978-3-031-31108-6_2)
11. Lopez-Martin, M., Carro, B., Sanchez-Esguevillas, A., Lloret, J.: Conditional variational autoencoder for prediction and feature recovery applied to intrusion detection in iot. *Sensors* **17**(9), 1967 (2017). <https://doi.org/10.3390/s17091967>
12. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems* **30** (2017)
13. Nguyen, T.D., Marchal, S., Miettinen, M., Fereidooni, H., Asokan, N., Sadeghi, A.R.: DIoT: A federated self-learning anomaly detection system for IoT. In: Proc. IEEE 39th Int. Conf. on Distributed Computing Systems (ICDCS). pp. 756–767 (2019). <https://doi.org/10.1109/ICDCS.2019.00080>
14. NVIDIA Corporation: Jetson orin nano super developer kit (2024), <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/nano-super-developer-kit/>
15. Ruff, L., et al.: A unifying review of deep and shallow anomaly detection. *Proceedings of the IEEE* **109**(5), 756–795 (2021)
16. Sforzin, A., Gómez Mármol, F., Conti, M., Bohli, J.M.: RPiDS: Raspberry pi IDS — a fruitful intrusion detection system for IoT. In: Proc. Int. IEEE Conf. on Ubiquitous Intelligence & Computing (UIC/ATC/ScalCom). pp. 440–448 (2016). <https://doi.org/10.1109/UIC-ATC-ScalCom-CBDCoM-IoP-SmartWorld.2016.0080>
17. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* pp. 108–116 (2018)
18. Shi, W., et al.: Edge computing: Vision and challenges. In: *IEEE Internet of Things Journal*. vol. 3, pp. 637–646 (2016)
19. Vinayakumar, R., Alazab, M., Soman, K.P., Poornachandran, P., Al-Nemrat, A., Venkatraman, S.: Deep learning approach for intelligent intrusion detection system. *IEEE Access* **7**, 41525–41550 (2019)
20. Zaharia, M., et al.: Spark: Cluster computing with working sets. *HotCloud* (2010)